

Xử lý khi Validation Loss không chịu giảm

- Đôi khi trong quá trình huấn luyện, Loss hoàn toàn không giảm và độ chính xác giữ nguyên ở ngưỡng Random Guess (chọn bừa).
- Điều này thường liên quan đến **Learning Rate** (Tốc độ học) đang quá cao hoặc quá thấp.
- Ví dụ: Tốc độ học khổng lồ '1.0' sẽ khiến các bước Gradient bay nhảy hỗn loạn trong không gian hàm mất mát.

```
1 # Tình huống khiến mô hình "chết lâm sàng", Accuracy không thể tăng
2 model.compile(
3     optimizer=keras.optimizers.RMSprop(learning_rate=1.0),
4     loss="sparse_categorical_crossentropy",
5     metrics=["accuracy"],
6 )
```

Điều chỉnh Learning Rate và Batch Size

- Khi hạ Learning Rate xuống mức tiêu chuẩn (ví dụ '1e-2', '1e-3'), Loss sẽ bắt đầu giảm trở lại.
- Nếu mạng vẫn không học, thử Tăng Batch Size. Lô dữ liệu lớn hơn sẽ tạo ra các vector đạo hàm ít nhiễu hơn, cung cấp định hướng chính xác hơn.

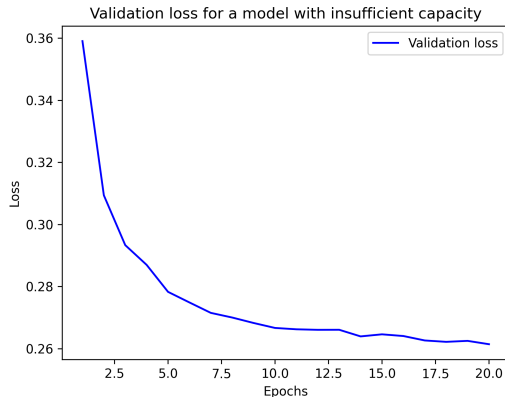
```
1 # Giảm tốc độ học giúp mô hình hội sinh
2 model.compile(
3     optimizer=keras.optimizers.RMSprop(learning_rate=1e-2),
4     loss="sparse_categorical_crossentropy",
5     metrics=["accuracy"],
6 )
7 model.fit(train_images, train_labels, epochs=10, batch_size=128)
```

Dung lượng Mô hình (Model Capacity) là gì?

- Dung lượng của mạng nơ-ron tỷ lệ thuận với số lượng tham số (trọng số) mà nó sở hữu. Số lượng layer và số unit quyết định đại lượng này.
- **Dung lượng quá nhỏ:** Không đủ khả năng "ghi nhớ" các quy luật biểu diễn phi tuyến tính phức tạp → Underfitting.
- **Dung lượng quá lớn:** Quá dư thừa, mạng dễ dàng học vẹt (như một cuốn từ điển mapping từng input ra output) → Overfitting cực nhanh.
- Mục tiêu của người học sâu là thiết kế mạng có dung lượng vừa khít, có khả năng giảm loss tối đa nhưng trì hoãn việc overfit muộn nhất.

Khi Dung lượng Mô hình Không Đủ (Insufficient)

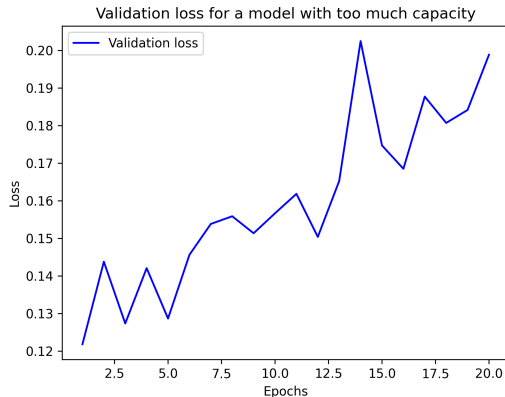
- Thử nghiệm dùng mạng hồi quy Logistic quá nhỏ (ví dụ cấu trúc nông chỉ xuất thẳng ra softmax) trên MNIST.
- Kết quả: Validation Loss không bao giờ xuống tới mức thấp mong muốn. Loss cứ lặp lại ở mức 0.26 và giữ nguyên (Stalled).
- Hiện tượng này xác nhận mạng không đủ sức mạnh để khái quát không gian giả thuyết phức tạp của hình ảnh.



Hình 5.10: Loss khi Model Capacity quá yếu

Khi Dung lượng Mô hình Quá Lớn (Excessive)

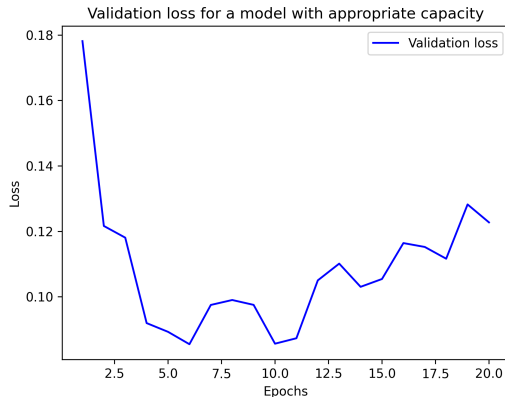
- Thử nghiệm dùng mạng khổng lồ (3 lớp ẩn, mỗi lớp 2048 nơ-ron - hàng chục triệu tham số).
- Kết quả: Do sức mạnh học vẹt quá kinh khủng, nó nhớ toàn bộ tập Training trong vài giây. Validation Loss chạm đáy ngay tại Epoch 1, sau đó vọt lên mạnh mẽ.
- Đường Validation Loss cũng rất nhiều (noise) chứng tỏ sự mất ổn định.



Hình 5.11: Loss khi Model Capacity quá lớn

Dung lượng Mô hình Lý Tưởng (Correct Capacity)

- Mô hình có 2 lớp ẩn (mỗi lớp 128 units).
- Validation Loss giảm từ từ và duy trì ở mức tối thiểu trước khi bắt đầu có dấu hiệu tăng nhẹ ở epoch thứ 8.
- Đây là thiết kế kiến trúc (Architecture) cân bằng tuyệt vời giữa tối ưu hóa và khái quát hóa dữ liệu hình chữ số MNIST.



Hình 5.12: Loss khi Model Capacity phù hợp

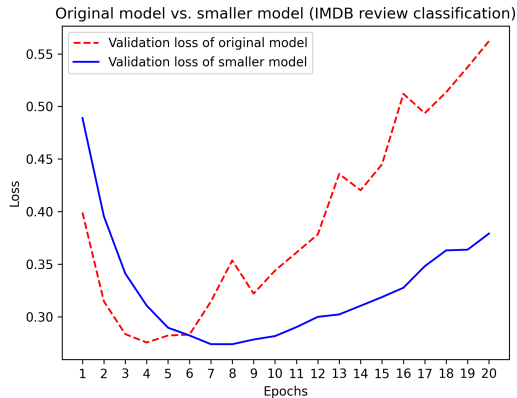
So sánh trên bài toán IMDB (Phân tích cảm xúc)

- Mạng phân tích phim gốc ở Chương 4 sử dụng 2 lớp, 16 units.
- Hãy thử nghiệm giảm xuống còn 4 units, và tăng vọt lên 512 units.

```
1 # Mạng IMDB gốc (16 units)
2 model_base = keras.Sequential([
3     layers.Dense(16, activation="relu"),
4     layers.Dense(16, activation="relu"),
5     layers.Dense(1, activation="sigmoid")
6 ])
7
8 # Mạng IMDB nhỏ (4 units) -> Mục tiêu giảm thiểu Overfitting
9 model_small = keras.Sequential([
10     layers.Dense(4, activation="relu"),
11     layers.Dense(4, activation="relu"),
12     layers.Dense(1, activation="sigmoid")
13 ])
```

Trực quan: Mạng Gốc (16 units) vs Mạng Nhỏ Hơn (4 units)

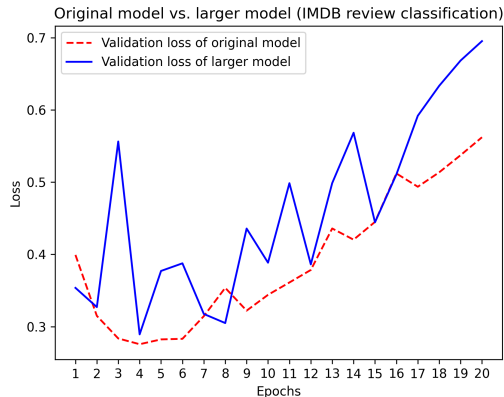
- Đường có dấu chấm tròn (Dots) đại diện cho Mạng gốc 16 units.
- Đường đứt khúc (Solid/Crosses) đại diện cho Mạng nhỏ 4 units.
- Phân tích: Mạng nhỏ (Dung lượng thấp) phải mất nhiều epoch hơn (6 epoch) để bắt đầu có dấu hiệu tăng Validation Loss. Khi Overfit, đồ thị cũng dốc thoải chậm hơn rất nhiều so với bản gốc.
- Kết luận: Giảm dung lượng mạng là kỹ thuật Regularization gốc rễ.



Hình 5.13: Mạng nhỏ (Crosses) vs Mạng gốc (Dots)

Trực quan: Mạng Gốc (16 units) vs Mạng Lớn Hơn (512 units)

- Đường có dấu chấm tròn (Dots) đại diện cho Mạng gốc 16 units.
- Đường đứt khúc đại diện cho Mạng 512 units.
- Phân tích: Mạng 512 units rơi vào tình trạng "Ghi nhớ thần tốc". Ngay từ epoch đầu tiên nó đã Overfit dữ liệu và vọt lên rất cao, biên độ của Loss bị nhiễu lớn.



Hình 5.14: Mạng lớn (Crosses) vs Mạng gốc (Dots)

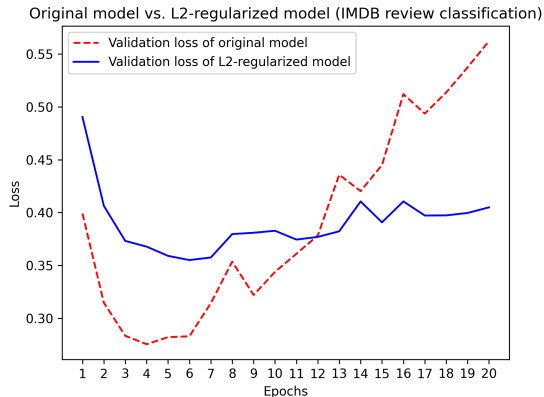
Weight Regularization (Phạt Trọng Số L1 / L2)

- Áp dụng nguyên lý Ockham's razor: Lời giải thích đơn giản hơn thường có xu hướng đúng. Một mô hình có các trọng số siêu nhỏ phân bố đều thì đơn giản hơn một mô hình chứa các trọng số khổng lồ bất thường.
- **L1 Regularization:** Thêm mức phạt tỷ lệ với trị tuyệt đối của trọng số $|W|$. Sẽ ép nhiều trọng số về đúng bằng 0 (Tạo sự thưa thớt - Sparsity).
- **L2 Regularization (Weight Decay):** Thêm mức phạt tỷ lệ thuận với bình phương trọng số W^2 . Ép trọng số nhỏ về tiệm cận 0.

```
1 from keras.regularizers import l2, l1_l2
2
3 # Thêm tham số kernel_regularizer để tích hợp Phạt trọng số
4 model = keras.Sequential([
5     layers.Dense(16, kernel_regularizer=l2(0.002), activation="relu"),
6     layers.Dense(16, kernel_regularizer=l2(0.002), activation="relu"),
7     layers.Dense(1, activation="sigmoid")
8 ])
```

Hiệu quả của Phạt trọng số L2

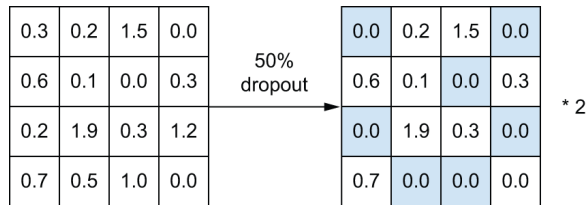
- Tham số ' $\lambda(0.002)$ ' sẽ cộng thêm $0.002 \times W^2$ vào hàm Loss gốc. Lưu ý, khoản phạt này chỉ tác động lúc Huấn luyện, tắt hoàn toàn lúc Kiểm tra.
- Biểu đồ cho thấy mạng dùng L2 (Crosses) duy trì Validation Loss ở mức an toàn và lâu overfit hơn đáng kể so với mạng không phạt.
- Thường được dùng tốt nhất trên các mô hình có kích cỡ bé và vừa.



Hình 5.15: Hiệu quả của L2 Regularization (Crosses)

Kỹ thuật Dropout (Ngắt Nơ-ron Ngẫu nhiên)

- Kỹ thuật cách mạng do **Geoffrey Hinton** phát minh.
- Cơ chế: Trong từng bước tối ưu, tại một layer, đột ngột vô hiệu hóa ngẫu nhiên (ép về giá trị 0) một tỷ lệ (rate) các nơ-ron, thường là $0.2 \sim 0.5$.
- Ngăn chặn hiện tượng "đồng thích nghi" (Co-adaptation): Các nơ-ron không thể ỷ lại, dựa dẫm học chung một tính năng với nơ-ron khác → Buộc phải tự hình thành những đặc trưng độc lập và mạnh mẽ.



Hình 5.16: Quá trình Dropout tại thời điểm Training

Cơ chế cân bằng Toán học của Dropout

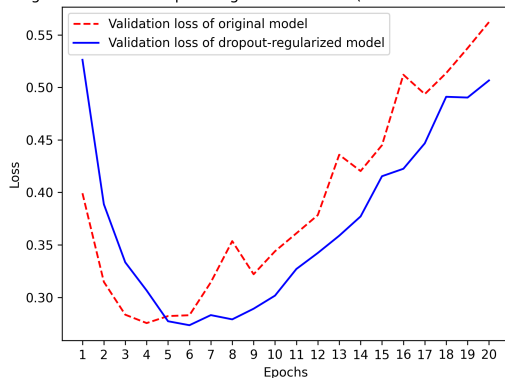
- **Khi Training:** Ngắt 50% nơ-ron (rate = 0.5).
- **Khi Testing (Inference):** Tuyệt đối không Dropout! Mô hình chạy full công suất.
- Vấn đề: Nếu lúc học có 50 người kéo co, lúc thi có tận 100 người kéo co thì tổng lực (tổng Activation) bị tăng gấp đôi, làm vỡ hàm Loss.
- Giải pháp Keras: Tại lúc học, các kích hoạt còn sống sót sẽ được **Scale Up** (nhân cho $\frac{1}{1-\text{rate}} = 2$).

```
1 model = keras.Sequential([
2     layers.Dense(16, activation="relu"),
3     layers.Dropout(0.5), # Nhét lớp Dropout ngay sau lớp Dense
4     layers.Dense(16, activation="relu"),
5     layers.Dropout(0.5),
6     layers.Dense(1, activation="sigmoid")
7 ])
```

Hiệu quả của Dropout

- Trong biểu đồ này, mạng sử dụng Dropout rate = 0.5 (Crosses) đã kháng cự lại sự Overfit vô cùng kiên cường.
- Đây chính là kỹ thuật Regularization hiệu quả và được áp dụng gần như mặc định cho các mô hình Deep Learning cực sâu hiện tại.

Original model vs. dropout-regularized model (IMDB review classification)



Hình 5.17: Hiệu quả của Dropout (Crosses)

Kỹ thuật tính năng (Feature Engineering)

- Feature Engineering là quá trình sử dụng kiến thức, quy luật chuyên gia (Domain Knowledge) để biến đổi dữ liệu đầu vào sao cho nó trở nên thân thiện nhất với thuật toán ML/DL.
- DL hiện đại có khả năng trích xuất tính năng ẩn một cách tự động, tuy nhiên với các tập dữ liệu nhỏ hoặc bài toán đặc thù, nếu ta thiết kế Feature Engineering tốt, mô hình sẽ tốn ít tài nguyên hơn, chạy nhanh hơn và chính xác hơn.
- Một bài toán kinh điển: Nhận diện thời gian trên đồng hồ kim.